

PROYECTO SNIFFER EN PYTHON

Javier Rincón Vivero

ÍNDICE

Contenido

Introducción	3
Arquitectura del programa.....	3
Decodificación de la cabecera IP	4
Decodificación de la cabecera UDP	5
Decodificación de la cabecera TCP.....	6
Decodificación del protocolo ICMP	7
Decodificación del protocolo DNS.....	8
Calculo y verificación de checksums	9
Pseudo-cabecera para UDP y TCP	9
Verificación.....	9
Pruebas y resultados experimentales	10
Captura de tráfico DNS.....	10
Captura de tráfico HTTP	11
Captura de tráfico ICMP	12
Análisis del Error en la Validación de Checksums	13
Código fuente completo.....	13

Introducción

En este documento se describe el desarrollo de un sniffer hecho en Python. Un sniffer es una aplicación capaz de capturar el tráfico que circula por una interfaz de red, y mediante el análisis de las cabeceras y datos de los paquetes, mostrar la información de los distintos protocolos.

En el código que se ha elaborado, he implementado decodificadores para los siguientes protocolos:

- Protocolo IP
- Protocolo UDP
- Protocolo TCP
- Protocolo ICMP
- Protocolo DNS
- Protocolo HTTP
- Protocolo SMTP
- Protocolo FTP

También, se ha incluido la verificación de los checksums de IP, UDP, TCP e ICMP.

Arquitectura del programa

El código de este sniffer está compuesto de una parte principal y de un conjunto de funciones auxiliares. La parte principal está compuesta de la decodificación de la cabecera IP. Las funciones auxiliares decodifican el resto de protocolos.

Funciones implementadas:

- BytesAWords(L8): convierte una lista de bytes en una lista de palabras de 16 bits, añadiendo un byte de relleno si la longitud es impar.
- calcular_checksum(palabras): calcula el checksum a 16 bits con complemento a 1, utilizado por todos los protocolos que requieren verificación.
- obtener_cabecera_ip(datos_recibidos): decodifica la cabecera IP, calcula su checksum y realiza la decodificación del protocolo de transporte correspondiente.
- obtener_cabecera_udp(bytes_ip_origen, bytes_ip_destino, payload): decodifica la cabecera UDP y verifica su checksum mediante la pseudo-cabecera
- obtener_cabecera_tcp(bytes_ip_origen, bytes_ip_destino, payload): decodifica la cabecera TCP, sus flags, sus opciones y verifica el checksum con la pseudo-cabecera.

- `interpretar_tipo_codigo_icmp(tipo_icmp, codigo)`: interpreta los tipos y códigos ICMP y muestra el mensaje correspondiente.
- `obtener_cabecera_icmp(payload)`: decodifica el mensaje ICMP y verifica su checksum.
- `leerNombreDNS(mensaje, posicion)`: extrae nombres de dominio gestionando la compresión por punteros del protocolo DNS.
- `procesar_mensaje_dns(payload)`: decodifica la cabecera y los registros del servicio DNS.
- `decodificar_protocolo(id_protocolo, bytes_ip_origen, bytes_ip_destino, payload)`: función que invoca el decodificador adecuado según el campo Protocolo de la cabecera IP.
- `main()`: configura el socket en modo promiscuo, captura 100 paquetes y los procesa secuencialmente.

Esquema de ejecución:

La función `main()` configura un raw socket, lo asocia a la IP local, activa la inclusión de cabeceras IP, habilita la recepción de todos los paquetes. Después, delega su decodificación en la función `obtener_cabecera_ip(...)`, que invoca al decodificador específico del protocolo de transporte correspondiente. Cuando se procesan 100 paquetes, se desactiva el modo promiscuo y el socket se cierra.

Decodificación de la cabecera IP

La cabecera IP tiene una longitud mínima de 20 bytes. Para decodificarla se ha utilizado el struct, que permite desempaquetar datos binarios en variables tipadas. En este caso el formato ha sido `!BBHHBBH4s4s`, que tiene el siguiente significado:

Símbolo	Tipo	Campo IP
!	Big-endian	Orden de bytes para redes
B	1 byte	Versión + IHL
B	1 byte	Tipo de servicio (TOS)
H	2 bytes	Longitud total
H	2 bytes	Identificación
H	2 bytes	Flags + Offset de fragmento
B	1 byte	TTL

B	1 byte	Protocolo
H	2 bytes	Checksum de cabecera
4s	4 bytes	IP origen
4s	4 bytes	IP destino

Decodificación de la cabecera UDP

La cabecera UDP tiene una longitud fija de 8 bytes, por lo que su decodificación es más sencilla que la de IP. Para desempaquetarla se ha utilizado el formato !HHHH del módulo struct, que tiene el siguiente significado:

Símbolo	Tipo	Campo UDP
!	Big-endian	Orden de bytes para redes
H	2 bytes	Puerto origen
H	2 bytes	Puerto destino
H	2 bytes	Longitud total del datagrama
H	2 bytes	Checksum

El checksum se verifica construyendo la pseudo-cabecera con las direcciones IP, el identificador de protocolo y la longitud del segmento, y aplicando sobre el conjunto el algoritmo del complemento a 1. Si el campo checksum llega a cero, se considera que el origen no lo ha calculado y no se realiza la verificación.

Una vez decodificada la cabecera, si los puertos origen o destino son el 53 se delega el procesamiento del payload en la función `procesar_mensaje_dns`, ya que ese puerto corresponde al servicio DNS. En cualquier otro caso, los datos UDP se muestran tal cual.

Decodificación de la cabecera TCP

La cabecera TCP tiene una longitud mínima de 20 bytes, aunque puede ser mayor si incluye opciones. Para decodificarla se utiliza el formato !HLLHHHH del módulo struct, que tiene el siguiente significado:

Símbolo	Tipo	Campo TCP
!	Big-endian	Orden de bytes para redes
H	2 bytes	Puerto origen
H	2 bytes	Puerto destino
L	4 bytes	Número de secuencia
L	4 bytes	Número de asentimiento
H	2 bytes	Offset + Reservado + Flags
H	2 bytes	Tamaño de ventana
H	2 bytes	Checksum
H	2 bytes	Puntero urgente

El campo de 16 bits que contiene Offset, Reservado y Flags se descompone mediante operaciones de desplazamiento y máscaras. El offset indica la longitud de la cabecera en palabras de 32 bits, mientras que los 6 bits de menor peso corresponden a los flags URG, ACK, PSH, RST, SYN y FIN. Cada uno de estos flags se extrae de manera individual mediante un desplazamiento y una operación AND.

Si el offset indica un valor mayor de 20 bytes, los bytes adicionales corresponden a las opciones TCP, que también se muestran. Igual que en UDP, el checksum se verifica construyendo la pseudo-cabecera y aplicando el algoritmo del complemento a 1.

Una vez decodificada la cabecera, se examinan los puertos para identificar protocolos de aplicación: si alguno de ellos es el 80 los datos se muestran como HTTP, si es el 25 como SMTP y si es el 21 como FTP. En estos casos los datos se

decodifican como texto UTF-8 ignorando los bytes que no se puedan interpretar.

Decodificación del protocolo ICMP

El protocolo ICMP se utiliza para enviar mensajes de control y de error entre los equipos de la red. La cabecera ICMP tiene una longitud fija de 8 bytes y se ha decodificado utilizando el formato !BBHI del módulo struct, que tiene el siguiente significado:

Símbolo	Tipo	Campo ICMP
!	Big-endian	Orden de bytes para redes
B	1 byte	Tipo
B	1 byte	Código
H	2 bytes	Checksum
I	4 bytes	Resto de la cabecera

El significado del campo "Resto de la cabecera" depende del tipo de mensaje. Por ejemplo, en las peticiones y respuestas de eco contiene el identificador y el número de secuencia, mientras que en los mensajes de error suele estar a cero. La verificación del checksum se hace recorriendo todo el mensaje ICMP con el algoritmo del complemento a 1, sin necesidad de pseudo-cabecera. Una vez decodificada la cabecera, la función `interpretar_tipo_codigo_icmp` se encarga de identificar el significado del mensaje a partir de los campos Tipo y Código. Los principales mensajes que se reconocen son los siguientes: respuesta de eco, petición de eco, tiempo excedido, destino inalcanzable, disminución del origen, redirección, problema de parámetros y solicitud y respuesta de marca de tiempo. Para los tipos 11, 3 y 5 se interpreta además el campo Código, que aporta información más detallada sobre la causa del mensaje.

Decodificación del protocolo DNS

El protocolo DNS se transporta normalmente sobre UDP en el puerto 53, por lo que su decodificación se invoca cuando alguno de los puertos de la cabecera UDP coincide con ese valor. La cabecera DNS tiene una longitud fija de 12 bytes y se desempaqueta con el formato !HHHHHH del módulo struct, que tiene el siguiente significado:

Símbolo	Tipo	Campo DNS
H	2 bytes	Transaction ID
H	2 bytes	Flags (QR, OpCode, AA, TC, RD, RA, RCode)
H	2 bytes	Número de consultas (QDCOUNT)
H	2 bytes	Número de respuestas (ANCOUNT)
H	2 bytes	Registros de autoridad (NSCOUNT)
H	2 bytes	Registros adicionales (ARCOUNT)

El campo Flags se descompone en sus campos individuales mediante desplazamientos y máscaras. El bit QR indica si el mensaje es de consulta o de respuesta, OpCode el tipo de operación, AA si la respuesta es autoritativa, TC si el mensaje fue truncado, RD si se solicita recursión, RA si el servidor admite recursión, y RCode el código de retorno.

Tras la cabecera viene un número variable de secciones de consulta y respuesta. Para procesarlas se ha implementado la función leerNombreDNS, que extrae los nombres de dominio teniendo en cuenta el mecanismo de compresión por punteros del protocolo DNS. Cuando el byte de longitud de una etiqueta tiene los dos bits superiores a 1, se interpreta como un puntero hacia otra posición del mensaje. La función mantiene un contador de iteraciones como protección frente a posibles bucles maliciosos.

Para cada consulta se muestra el nombre de dominio, el tipo de registro y la clase. Los tipos reconocidos son A, NS, CNAME, SOA, PTR, MX, TXT y AAAA. En las respuestas se incluye además el TTL y el valor del registro: para los registros A se interpreta como una dirección IPv4, para los AAAA como una dirección IPv6, para los NS, CNAME y PTR como otro nombre de dominio, y en el resto de casos se muestra el contenido en bruto.

Calculo y verificación de checksums

Tanto la cabecera IP como las cabeceras UDP, TCP e ICMP incluyen un campo de checksum que permite detectar si ha habido errores en la transmisión. Este calculo se realiza siempre mediante el mismo algoritmo:

```
def calcular_checksum(palabras):  
    suma = sum(palabras)  
    while suma >> 16:  
        suma = (suma & 0xFFFF) + (suma >> 16)  
    return ~suma & 0xFFFF
```

Los pasos del algoritmo son los siguientes: en primer lugar, se suman todas las palabras de 16 bits del bloque de datos sobre el que se quiere calcular el checksum. A continuación, mientras el resultado tenga bits por encima de los 16 bits inferiores (es decir, mientras haya acarreo), se suman los 16 bits superiores a los 16 bits inferiores, repitiendo este proceso hasta que no quede ningún acarreo. Finalmente, se aplica el complemento a 1 al resultado, quedándose únicamente con los 16 bits inferiores mediante la operación `~suma & 0xFFFF`, que es el valor que se inserta en el campo checksum del paquete.

Pseudo-cabecera para UDP y TCP

Los protocolos UDP y TCP calculan su checksum no solo sobre su propia cabecera, sino también sobre una pseudo cabecera construida por información de la cabecera IP. Esta cabecera contiene la IP de origen (bytes), IP de destino(4 bytes), un byte a cero, el byte de protocolo y una longitud del segmento (2 bytes)

```
struct.pack('!4s4sBBH', bytes_ip_origen, bytes_ip_destino, 0, 17,  
len(payload))
```

Verificación

Para verificar un checksum recibido, basta con aplicar de nuevo el mismo algoritmo de cálculo sobre el bloque de datos completo, esta vez incluyendo el propio campo checksum tal y como llegó en el paquete (sin ponerlo a cero). Si la suma resultante, una vez plegados los acarreos y aplicado el complemento a 1, da como resultado 0x0000, el checksum es válido y el paquete no ha sufrido alteraciones detectables durante la transmisión; en caso contrario, se considera que el paquete está corrupto. En el código del sniffer este criterio se aplica directamente comparando el resultado de `calcular_checksum` con cero, mostrando entonces el mensaje "valido" o "erroneo" según corresponda.

Pruebas y resultados experimentales

Para verificar que el sniffer funcione correctamente se han probado distintas situaciones de envío y recepción de tráfico, capturando los paquetes y comprobando que la decodificación coincide con lo esperado:

Captura de tráfico DNS

```
=====
===== Paquete 97 =====
=====
*****INTERNET PROTOCOL*****
Version IP = 4
IHL = 20 bytes
TOS = 0
Longitud Total = 137 bytes
Identificacion = 55960
Flags = 0b10
  Don't Fragment
Desplazamiento Fragmento = 0
TTL = 64
Protocolo = 17
Checksum = 56421 (valido)
IP origen = 192.168.1.1
IP destino = 192.168.1.20
*****USER DATAGRAM PROTOCOL*****
Puerto origen = 53
Puerto destino = 63897
Longitud = 117 bytes
Checksum = 2503 (valido)
*****DOMAIN NAME SYSTEM*****
Transaction ID = 0x8550
QR = 1, mensaje de respuesta
  OpCode: 0, AA: 0, TC: 0, RD: 1, RA: 1, RCode: 0
N consultas = 1
N respuestas = 3
N autoridad = 0
N adicionales = 0
Consulta 1: pbs.twimg.com Tipo = A Clase = 1
Respuesta 1: pbs.twimg.com Tipo = CNAME TTL = 695 Valor = pbs.twimg.com.cdn.cloudflare.net
Respuesta 2: pbs.twimg.com.cdn.cloudflare.net Tipo = A TTL = 96 Valor = 172.64.150.129
Respuesta 3: pbs.twimg.com.cdn.cloudflare.net Tipo = A TTL = 96 Valor = 104.18.37.127
```

En esta captura se puede ver que el paquete numero 97 representa un mensaje de respuesta DNS recibido desde 192.168.1.1 en el puerto 53 hacia 192.168.1.20 (mi ordenador) con el puerto 63897, encapsulado en un datagrama UDP transportado en un paquete IPv4. La cabecera IP indica efectivamente el protocolo 17 (UDP), longitud total de 137 bytes, TTL= 64, y el flag DF activo. El checksum de IP es validado correctamente, recalculando el algoritmo sobre la cabecera IP con el campo del checksum puesto a 0.

A nivel de UDP, la longitud del datagrama es de 117 bytes y el checksum también se valida correctamente sobre la pseudo cabecera (direcciones IP de origen y destino, un byte a cero, el byte de protocolo 17, y la longitud UDP) concatenada con la cabecera y los datos UDP. Como el puerto de origen es 53, el código delega el procesamiento del payload a la función `procesa_mensaje_dns(...)`.

A nivel de DNS, el Transaction ID es 0x8550 y los flags activos son QR(respuesta), RD (recursión solicitada por el cliente) y RA (recursión disponible en el servidor). En esta cabecera se puede comprobar que hay una consulta y tres respuestas. La pregunta es por el nombre pbs.twimg.com de tipo A y clase 1. En la primera respuesta se puede ver que es un CNAME que redirige pbs.twimg.com a pbs.twimg.com.cdn.cloudflare.net, las dos siguientes respuestas asocian ese alias con las direcciones 172.64.150.129 y 104.18.37.127.

Captura de tráfico HTTP

```
=====
===== Paquete 10 =====
=====
*****INTERNET PROTOCOL*****
Version IP = 4
IHL = 20 bytes
TOS = 0
Longitud Total = 40 bytes
Identificacion = 40714
Flags = 0b10
  Don't Fragment
Desplazamiento Fragmento = 0
TTL = 128
Protocolo = 6
Checksum = 39377 (valido)
IP origen = 192.168.1.20
IP destino = 188.184.67.127
*****TRANSMISSION CONTROL PROTOCOL*****
Puerto origen = 61525
Puerto destino = 80
Numero secuencia = 1299585821
Numero asentimiento = 1268682865
Longitud cabecera = 20 bytes
Flags:
  URG: 0, ACK: 1, PSH: 0, RST: 0, SYN: 0, FIN: 0
Ventana = 255
Checksum = 0xc20e (erroneo)
Puntero urgente = 0
Datos TCP = b''
--- HTTP ---
```

En esta captura se puede ver que el paquete numero 10 representa un segmento TCP saliente desde mi ordenador (192.168.1.20) hacia el servidor 188.184.67.127 en el puerto 80, lo que confirma que se trata de trafico HTTP. En la cabecera se confirma efectivamente que se trata del protocolo 6 (TCP), longitud total de 40 bytes, TTL= 128,y el flag DF activo. También es interesante comprobar que como el datagrama IP ocupa 40 bytes, el IHL es de 20 bytes, y el tamaño del segmento TCP tiene también 20 bytes, el tamaño de este segmento es el mínimo por lo que no transporta datos de aplicación, como se puede comprobar (Datos TCP= b'').

A nivel TCP, el único flag activo es ACK, lo que confirma que el cliente esta confirmando al servidor la recepción de bytes hasta el numero de asentimiento 1268682865. La ventana anunciada es 255 bytes, la longitud de cabecera es de 20 bytes, y el puntero urgente es 0.

Es importante destacar que el checksum TCP aparece como erroneo. Esto no es debido a un fallo del código, es debido al TCP Checksum Offload. Se comentara sobre este fenómeno en la página 12

Captura de tráfico ICMP

```
=====
===== Paquete 33 =====
=====
*****INTERNET PROTOCOL*****
Version IP = 4
IHL = 20 bytes
TOS = 0
Longitud Total = 60 bytes
Identificacion = 534
Flags = 0b0
Desplazamiento Fragmento = 0
TTL = 128
Protocolo = 1
Checksum = 53637 (valido)
IP origen = 192.168.1.20
IP destino = 216.58.205.46
*****INTERNET CONTROL MESSAGE PROTOCOL*****
Tipo = 8
Codigo = 0
Checksum = 0x4d54 (valido)
Resto cabecera = 0x10007
Mensaje: Peticion de eco
Datos = b'abcdefghijklmnpqrstuvwabcdefghi'
```

En esta captura se puede ver que el paquete numero 33 representa una peticion de eco ICMP saliente de mi ordenador (192.168.1.20) hacia el servidor 216.58.205.46 (un servidor de Google). La cabecera IP indica efectivamente procolo 1 (ICMP), longitud total de 60 bytes, TTL = 128, y a diferencia de los paquetes anteriores, todos los flags están a cero, lo cual es habitual en peticiones ICMP. EL checksum IP se valida correctamente.

A nivel ICMP, el campo tipo es 8 y el de código es 0, lo que la función interpretar_tipo_codigo_icmp(...) identifica correctamente como Peticion de eco. El checksum se valida correctamente sobre el mensaje ICMP completo, sin pseudo cabeceras, ya que en el caso de ICMP no se utiliza. El campo Resto cabecera tiene valor 0x10007, donde 1 representa el proceso ping y 0007 corresponden al número de secuencia de la petición. Los datos transportados

son los bytes de relleno característicos que el comando ping en Windows envía por defecto.

Análisis del Error en la Validación de Checksums

Durante la realización de estas capturas de pantalla al hacer las pruebas, he observado que una gran cantidad de paquetes presentaban un estado de checksum erróneo cuando la dirección IP de origen coincidía con la interfaz local del host donde ejecutaba el sniffer. Sin embargo los entrantes tenían el checksum como válido.

Este fenómeno se debe a una técnica de optimización de hardware conocida como Checksum Offloading. Es una técnica de optimización donde el sistema operativo delega el cálculo y verificación de sumas de comprobación a la tarjeta de red.

Esto libera carga de trabajo a la CPU, mejorando el rendimiento general del sistema al procesar el tráfico de red mediante hardware especializado.

Código fuente completo

A continuación, se incluye el código fuente íntegro del sniffer:

```
import socket
import struct
def BytesAWords(L8):
    L16 = []
    if len(L8)%2 != 0:
        L8.append(0)
    for i in range(0, len(L8), 2):
        L16.append(L8[i]*256 + L8[i+1])
    return L16

def calcular_checksum(palabras):
    suma = sum(palabras)
    while suma >> 16:
        suma = (suma & 0xFFFF) + (suma >> 16)
    return ~suma & 0xFFFF

def obtener_cabecera_ip(datos_recibidos):
    print("*****INTERNET PROTOCOL*****")

    if len(datos_recibidos) < 20:
```

```

        print("Paquete demasiado corto para ser IPv4, descartado")
        return

version_check = datos_recibidos[0] >> 4
if version_check != 4:
    print(f"Paquete no IPv4 (version = {version_check}), descartado")
    return
valores_desempaquetados = struct.unpack('!BBHHHBBH4s4s',
datos_recibidos[:20])

version_ihl = valores_desempaquetados[0]
version = version_ihl >> 4
ihl = version_ihl & 0x0F
tamano_cabecera = ihl * 4

tipo_servicio, longitud_total, id_paquete, flags_offset, ttl,
protocolo, checksum = valores_desempaquetados[1:8]

ip_origen_bytes = valores_desempaquetados[8]
ip_destino_bytes = valores_desempaquetados[9]
ip_origen = socket.inet_ntoa(ip_origen_bytes)
ip_destino = socket.inet_ntoa(ip_destino_bytes)

flags = flags_offset >> 13
desplazamiento = flags_offset & 0x1FFF

ip_header_full = list(datos_recibidos[:ihl * 4])
ip_header_full[10] = 0
ip_header_full[11] = 0
words = BytesAWords(ip_header_full)
calculated_checksum = calcular_checksum(words)

print(f"Version IP = {version}")
print(f"IHL = {ihl * 4} bytes")
print(f"TOS = {tipo_servicio}")
print(f"Longitud Total = {longitud_total} bytes")
print(f"Identificacion = {id_paquete}")
print(f"Flags = {bin(flags)}")
if (flags & 0b010):
    print(" Don't Fragment")
if (flags & 0b001):
    print(" More Fragments")
print(f"Desplazamiento Fragmento = {desplazamiento}")
print(f"TTL = {ttl}")
print(f"Protocolo = {protocolo}")
print(f"Checksum = {checksum} ({'valido' if checksum ==
calculated_checksum else 'erroneo'})")
print(f"IP origen = {ip_origen}")
print(f"IP destino = {ip_destino}")

```

```

if ihl > 5:
    print(f'Opciones IP = {datos_recibidos[20:tamano_cabecera]}")

payload = datos_recibidos[tamano_cabecera:]

match protocolo:
    case 1 | 6 | 17:
        decodificar_protocolo(protocolo, ip_origen_bytes,
ip_destino_bytes, payload)
    case _:
        print(f"Datos IP = {payload}")

def obtener_cabecera_udp(bytes_ip_origen, bytes_ip_destino, payload):
    print("*****USER DATAGRAM PROTOCOL*****")
    cabecera_udp = payload[:8]
    puerto_origen, puerto_destino, longitud, checksum =
struct.unpack('!HHHH', cabecera_udp)

    pseudo_cabecera = struct.pack('!4s4sBBH', bytes_ip_origen,
bytes_ip_destino, 0, 17, len(payload))
    datos_checksum = list(pseudo_cabecera + payload)
    palabras = BytesAWords(datos_checksum)
    checksum_calculado = calcular_checksum(palabras)

    print(f"Puerto origen = {puerto_origen}")
    print(f"Puerto destino = {puerto_destino}")
    print(f"Longitud = {longitud} bytes")
    if checksum == 0:
        print(f"Checksum = {checksum} (no calculado)")
    else:
        estado_checksum = 'valido' if checksum_calculado == 0 else
'erroneo'
        print(f"Checksum = {checksum} ({estado_checksum})")

    if puerto_origen == 53 or puerto_destino == 53:
        procesar_mensaje_dns(payload[8:])
    else:
        print(f"Datos UDP = {payload[8:]}")

def obtener_cabecera_tcp(bytes_ip_origen, bytes_ip_destino, payload):
    print("*****TRANSMISSION CONTROL PROTOCOL*****")

    src_port, dst_port, seq, ack, offset_reserved_flags, window,
checksum, urg_ptr = struct.unpack('!HLLHHHH', payload[:20])

    offset = (offset_reserved_flags >> 12) * 4

```

```

flags = offset_reserved_flags & 0x3F

URG = (flags >> 5) & 1
ACK = (flags >> 4) & 1
PSH = (flags >> 3) & 1
RST = (flags >> 2) & 1
SYN = (flags >> 1) & 1
FIN = flags & 1

pseudo_cabecera = struct.pack('!4s4sBBH', bytes_ip_origen,
bytes_ip_destino, 0, 6, len(payload))
checksum_calculado =
calcular_checksum(BytesAWords(list(pseudo_cabecera + payload)))

opciones = payload[20:offset] if offset > 20 else b''
datos_tcp = payload[offset:]

print(f"Puerto origen = {src_port}")
print(f"Puerto destino = {dst_port}")
print(f"Numero secuencia = {seq}")
print(f"Numero asentimiento = {ack}")
print(f"Longitud cabecera = {offset} bytes")
print("Flags:")
print(f"  URG: {URG}, ACK: {ACK}, PSH: {PSH}, RST: {RST}, SYN: {SYN},
FIN: {FIN}")

print(f"Ventana = {window}")
estado_checksum = 'valido' if checksum_calculado == 0 else 'erroneo'
print(f"Checksum = {hex(checksum)} ({estado_checksum}")
print(f"Puntero urgente = {urg_ptr}")

if opciones:
    print(f"Opciones = {opciones}")
print(f"Datos TCP = {datos_tcp}")

if src_port == 80 or dst_port == 80:
    print("--- HTTP ---")
    print(datos_tcp.decode('utf-8', errors='ignore'))
elif src_port == 25 or dst_port == 25:
    print("--- SMTP ---")
    print(datos_tcp.decode('utf-8', errors='ignore'))
elif src_port == 21 or dst_port == 21:
    print("--- FTP ---")
    print(datos_tcp.decode('utf-8', errors='ignore'))

def interpretar_tipo_codigo_icmp(tipo_icmp, codigo):
    match tipo_icmp:
        case 0:

```



```

        print("Mensaje: Respuesta de eco")
    case 8:
        print("Mensaje: Peticion de eco")
    case 11:
        print("Mensaje: Tiempo excedido")
        match codigo:
            case 0: print("Mensaje: TTL excedido")
            case 1: print("Mensaje: Tiempo de reensamblaje de
fragmento excedido")
    case 3:
        print("Mensaje: Destino inalcanzable")
        match codigo:
            case 0: print("Mensaje: Red inalcanzable")
            case 1: print("Mensaje: Host inalcanzable")
            case 2: print("Mensaje: Protocolo inalcanzable")
            case 3: print("Mensaje: Puerto inalcanzable")
            case 4: print("Mensaje: Se requiere fragmentación, pero
DF=1")
            case 5: print("Mensaje: Error en la ruta de origen")
            case 6: print("Mensaje: Red de destino desconocida")
            case 7: print("Mensaje: Host de destino desconocido")
            case 8: print("Mensaje: Error de aislamiento del host de
origen")
            case 9: print("Mensaje: Acceso a la red prohibida")
            case 10: print("Mensaje: Acceso al host prohibido")
            case 11: print("Mensaje: Red inalcanzable para el TOS")
            case 12: print("Mensaje: Host inalcanzable para el TOS")
    case 4:
        print("Mensaje: Disminucion del origen")
    case 5:
        print("Mensaje: Redireccion")
        match codigo:
            case 0: print("Mensaje: Redireccion para la red")
            case 1: print("Mensaje: Redireccion para el host")
            case 2: print("Mensaje: Redireccion para TOS y red")
            case 3: print("Mensaje: Redireccion para TOS y host")
    case 12:
        print("Mensaje: Problema de parametros")
    case 13:
        print("Mensaje: Solicitud de marca de tiempo")
    case 14:
        print("Mensaje: Respuesta de marca de tiempo")
    case _:
        print(f"Tipo ICMP {tipo_icmp} desconocido")

def obtener_cabecera_icmp(payload):
    print("*****INTERNET CONTROL MESSAGE PROTOCOL*****")
    icmp_header = payload[:8]

```

```

icmp_type, code, checksum, rest = struct.unpack('!BBHI', icmp_header)

icmp_bytes = list(payload)
words = BytesAWords(icmp_bytes)
calculated_checksum = calcular_checksum(words)

print(f"Tipo = {icmp_type}")
print(f"Codigo = {code}")
print(f"Checksum = {hex(checksum)} ({'valido' if calculated_checksum
== 0 else 'erroneo'})")
print(f"Resto cabecera = {hex(rest)}")
interpretar_tipo_codigo_icmp(icmp_type, code)
print(f"Datos = {payload[8:]}")

def leerNombreDNS(mensaje, posicion):
    nombre = ''
    saltoRealizado = False
    posicionFinal = posicion
    iteraciones = 0
    while iteraciones < 128:
        if posicion >= len(mensaje):
            break
        longitud = mensaje[posicion]
        if longitud == 0:
            if not saltoRealizado:
                posicionFinal = posicion + 1
                break
        if (longitud & 0xC0) == 0xC0:
            if posicion + 1 >= len(mensaje):
                break
            puntero = ((longitud & 0x3F) << 8) | mensaje[posicion + 1]
            if not saltoRealizado:
                posicionFinal = posicion + 2
                saltoRealizado = True
            posicion = puntero
        else:
            posicion += 1
            if posicion + longitud > len(mensaje):
                break
            etiqueta = mensaje[posicion:posicion + longitud].decode('utf-
8', errors='ignore')
            if nombre == '':
                nombre = etiqueta
            else:
                nombre = nombre + '.' + etiqueta
            posicion += longitud
            iteraciones += 1
    return nombre, posicionFinal

```

```

def procesar_mensaje_dns(payload):
    print("*****DOMAIN NAME SYSTEM*****")
    if len(payload) < 12:
        print("Error: El mensaje DNS es demasiado corto")
        return

    transaction_id, flags, qdcount, ancount, nscount, arcount =
struct.unpack('!HHHHHH', payload[:12])

    QR = flags >> 15
    OpCode = (flags >> 11) & 0x0F
    AA = (flags >> 10) & 0x01
    TC = (flags >> 9) & 0x01
    RD = (flags >> 8) & 0x01
    RA = (flags >> 7) & 0x01
    RCode = flags & 0x0F

    print(f"Transaction ID = {hex(transaction_id)}")
    if QR == 0:
        print(f"QR = {QR}, mensaje de consulta")
    else:
        print(f"QR = {QR}, mensaje de respuesta")
    print(f" OpCode: {OpCode}, AA: {AA}, TC: {TC}, RD: {RD}, RA: {RA},
RCode: {RCode}")
    print(f"N consultas = {qdcount}")
    print(f"N respuestas = {ancount}")
    print(f"N autoridad = {nscount}")
    print(f"N adicionales = {arcount}")

    tiposDNS = {1: 'A', 2: 'NS', 5: 'CNAME', 6: 'SOA', 12: 'PTR', 15:
'MX', 16: 'TXT', 28: 'AAAA'}
    posicion = 12

    for i in range(qdcount):
        nombre, posicion = leerNombreDNS(payload, posicion)
        if posicion + 4 > len(payload):
            break
        tipo = (payload[posicion] << 8) + payload[posicion + 1]
        clase = (payload[posicion + 2] << 8) + payload[posicion + 3]
        posicion += 4
        print(f"Consulta {i + 1}: {nombre} Tipo = {tiposDNS.get(tipo,
tipo)} Clase = {clase}")

    for i in range(ancount):
        nombre, posicion = leerNombreDNS(payload, posicion)
        if posicion + 10 > len(payload):
            break

```

```

        tipo = (payload[posicion] << 8) + payload[posicion + 1]
        clase = (payload[posicion + 2] << 8) + payload[posicion + 3]
        ttl = struct.unpack('!L', payload[posicion + 4:posicion + 8])[0]
        rdlength = (payload[posicion + 8] << 8) + payload[posicion + 9]
        posicion += 10
        if posicion + rdlength > len(payload):
            break
        rdata = payload[posicion:posicion + rdlength]
        valor = ''
        if tipo == 1 and rdlength == 4:
            valor = socket.inet_ntoa(rdata)
        elif tipo == 28 and rdlength == 16:
            partes = []
            for j in range(0, 16, 2):
                partes.append(format((rdata[j] << 8) + rdata[j + 1],
'x'))
            valor = ':'.join(partes)
        elif tipo in (2, 5, 12):
            valor, _ = leerNombreDNS(payload, posicion)
        else:
            valor = str(rdata)
        posicion += rdlength
        print(f"Respuesta {i + 1}: {nombre} Tipo = {tiposDNS.get(tipo,
tipo)} TTL = {ttl} Valor = {valor}")

def decodificar_protocolo(id_protocolo, bytes_ip_origen,
bytes_ip_destino, payload):
    match id_protocolo:
        case 17:
            obtener_cabecera_udp(bytes_ip_origen, bytes_ip_destino,
payload)
        case 6:
            obtener_cabecera_tcp(bytes_ip_origen, bytes_ip_destino,
payload)
        case 1:
            obtener_cabecera_icmp(payload)
        case _:
            print("No se ha reconocido el protocolo")

def main():
    print("=====")
    print("SNIFFER JAVIER RINCON VIVERO")
    print("REDES DE COMPUTADORES")
    print("=====")

    HOST = '192.168.1.20'

```

```
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind((HOST, 0))
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

for i in range(100):
    print("=====")
    print(f"===== Paquete {i + 1} =====")
    print("=====")
    p = s.recvfrom(65565)
    datos = p[0]
    obtener_cabecera_ip(datos)

s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)

if __name__ == "__main__":
    main()
```